

Basic security — Switch It

University-project security write-up based on **what the repository actually does**. Remaining risks are stated honestly.

1. Threat model overview

Assets:

- User accounts and sessions
- Vehicle identity (plates)
- Parking locations
- Credits (virtual)
- Active handoff state / live location during a claim

Main attackers in scope:

- Another Switch It user trying to read or mutate someone else's data
- Race abuse on claims/credits
- Casual API misuse from the browser
- Collusion / fake handoffs / multi-account farming (harder; only partly mitigated)

Out of scope for MVP claims: nation-state attackers, perfect GPS anti-spoofing, legal parking enforcement.

2. Authentication

- Supabase Auth with **email + password** (primary product path).
- Browser session via `@supabase/ssr` cookies.
- `src/proxy.ts` refreshes session and redirects unauthenticated users away from protected prefixes (`/map` , `/spots/new` , `/profile` , `/history` , `/help` , `/onboarding` , ...). Authenticated users hitting `/login` , `/register` , or `/forgot-password` are redirected into the app. `/auth/reset-password` is **not** bounced away (recovery session must stay).
- Idle / resume: token refresh failure on live-location hooks returns `null` / idle and does **not** crash the route. A genuinely missing session is still a login redirect on the next authenticated server render — authentication is not weakened.
- `/auth/callback` exchanges PKCE `code` for a session (email confirmation **and** password recovery).
- Profile + starter credits are created once by the `auth.users` insert trigger (`handle_new_user`); confirmation, resend, and password reset do **not** re-run that bootstrap.

Email confirmation (signup)

- New accounts require **Confirm email** (Supabase Auth). Until confirmed, `signUp` returns no session.
- UI: **Check your email** with neutral copy — *"Check your inbox for a verification link to [email]." — plus "Already registered with this email? Sign in instead."*
- Unconfirmed login shows a friendly verify/resend state (not a raw Auth error). Resend uses neutral wording: *"If this email is awaiting verification, a new verification email has been sent."*

Anti-enumeration (signup + reset)

- With Confirm email enabled, `signUp` for an **already-registered** address often returns an obfuscated success (no error) so clients cannot probe whether an email exists. Switch It does **not** invent an “account exists” error from heuristics (including empty `identities`) and does **not** query `auth.users` from the app.
- Explicit Auth errors only (`user_already_exists` / “User already registered”, etc.) may show: “An account with this email already exists. Sign in instead.”
- Forgot-password success always uses neutral copy: “If an account exists for this email, you’ll receive a password reset link.”

Password policy

- Shared rules in `src/lib/auth/password-policy.ts` : 8+ chars, uppercase, lowercase, digit, special character, max 72.
- Applied on **Create Account** and **Set new password** (Forgot Password). Login does **not** re-apply signup policy locally.
- Supabase Auth remains the authoritative enforcer (Dashboard / local `config.toml` mirrored).

Forgot password (Login only)

There is **no** Change Password option in Profile.

Flow:

1. Login → **Forgot password?** → `/forgot-password`
2. `resetPasswordForEmail` with `redirectTo = {origin}/auth/callback/recovery`
3. User opens email → recovery callback exchanges code → `/auth/reset-password` (never onboarding/map)
4. `updateUser({ password })` under shared policy → sign out → **Password updated** → **Sign in**

The generic `/auth/callback` also treats `type=recovery` or `next=/auth/reset-password` as recovery when those params survive the Auth redirect.

Invalid/expired recovery links redirect to `/forgot-password?error=reset` (not the signup verification banner).

STUDY PRIORITY: Authentication proves *who the account/session is*; it does **not** prove a unique real-world person.

3. Authorization

Layers:

1. **Route gates** — must be logged in; often must have complete vehicle.
2. **RLS** — table policies limit SELECT/INSERT/UPDATE to permitted rows.
3. **RPC checks** — `SECURITY DEFINER` functions still verify `auth.uid()` (owner vs seeker).
4. **Column grants** — e.g. clients cannot directly UPDATE `profiles.credits` .

Concrete examples

Action	Who is allowed
Insert parking spot	Authenticated owner (<code>owner_id = auth.uid()</code>)
Claim spot	Authenticated seeker via <code>claim_spot</code> (not owner)

Complete claim	Spot owner/publisher via <code>complete_claim</code>
Cancel claim	Seeker of that claim
Cancel spot	Spot owner
Read credit ledger	Own transactions only
Read counterpart vehicle	Handoff participant via RPC (masked plate)
Read live-location snapshot	Claim participants only

4. Data privacy

Supported today:

- Counterpart sees a **masked** plate presentation; full plate is not the shared handoff UI path.
- Publisher verifies using last 2 digits without a spoken shared code chat UX.
- Live location is scoped to an **active handoff**:
 - Web and native: POST Edge Function `handoff-seeker-location` with the seeker JWT. The function checks `getUser + can_send_claim_location`, then service-role RPCs `upsert/throttle`, then private Broadcast **only if accepted**.
 - Ephemeral latest row in `claim_live_locations` (one row per claim, replaced; deleted on terminal). **Not** a permanent route history.
 - **Rate limiting**: authenticated seekers cannot flood Realtime with unlimited GPS or status updates.
- History returns the caller's terminal handoffs with visibility rules for addresses.

Still visible by design: approximate parking coordinates of available spots to seekers.

Do **not** claim coordinates are "never stored." Temporary snapshot storage exists on the native/Edge recovery path.

5. Database integrity

Important integrity tools:

- Partial unique: one active claim per spot / per seeker
- Partial unique: one debit and one credit ledger row per claim
- One open spot per owner
- Status CHECK constraints
- `profiles.license_plate` CHECK: `NULL` or digits-only length 5–8
(`profiles_license_plate_digits_allowed`)
- Cancellation reason/actor consistency checks
- Foreign keys from spots/claims to profiles

These prevent inconsistent states attackers might try to create via races.

License plates: Server Actions normalize separators to digits and validate length 5–8; PostgreSQL enforces the same canonical format. Duplicate plates across profiles are **intentionally allowed** (shared

family/company vehicles). Plate uniqueness is **not** treated as proof of human identity or as anti-Sybil protection.

6. Race-condition protection

- `SELECT ... FOR UPDATE` on spots during claim/start paths
 - `SELECT ... FOR UPDATE on claims` during live-location upserts (serializes first snapshot insert and concurrent GPS writers per claim)
 - Unique indexes as backstop
 - Idempotent completion + unique ledger rows
 - Same-seeker reclaim blocked after voluntary cancel
-

7. SECURITY DEFINER functions

Many handoff RPCs are `SECURITY DEFINER` with `search_path` locked down, granted to `authenticated`.

Why: complex multi-table updates need one transaction and cannot be safely expressed as naive client UPDATES under RLS alone.

Risk: if a DEFINER function forgets `auth.uid()` checks, it could bypass RLS.

Mitigation in this project: each sensitive RPC checks authentication and role (owner/seeker) before mutating; revoke execute from `anon` / `public` where migrations specify. Live-location write RPCs (`upsert_claim_live_location` , `try_accept_claim_location_status`) are **service_role only** — clients cannot call them directly; the Edge Function validates the seeker JWT first via `can_send_claim_location`.

STUDY PRIORITY: Be ready to open `claim_spot` / `complete_claim` and point to the `auth.uid()` checks.

8. Input validation

Layer	Examples
Zod in Server Actions	UUIDs, plate normalize/length 5–8, plate suffix shape, cancel reasons, publish fields
Client UX	Digit length, disable confirm until reason selected
DB CHECKS	Status enums, non-negative credits, vehicle year/color/type, license_plate NULL or <code>^[0–9]{5,8}\$</code>
RPC raises	Distance, expiry, reclaim, lockout

9. Secrets

Do not print real secret values in submission docs.

Variable	Exposure	Role
NEXT_PUBLIC_SUPABASE_URL	Public	Project URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Public	Browser/server user-scoped key (RLS applies)
NEXT_PUBLIC_MAPTILER_API_KEY	Public	Maps/geocoding
NEXT_PUBLIC_CARIMAGES_API_KEY	Public	CarImages JS loader
SUPABASE_SERVICE_ROLE_KEY	Server/Edge only	Bypasses RLS — must never be NEXT_PUBLIC_
.env.local	Local only	Not committed

`.env.example` lists placeholders for the core public Next vars.

Older docs mentioning `NEXT_PUBLIC_SUPABASE_ANON_KEY` are **stale**; code uses `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`.

10. Abuse / business-logic risks

Risk	Current mitigation	Remaining weakness	Possible future improvement (NOT CURRENTLY IMPLEMENTED)
Two users fake handoffs to farm credits	Plate verification + attempt lockout; credits only on complete	Colluding users who know each other's plates	Proximity-to-complete, repeated-pair detection, reputation
Multi-account / Sybil farming	New accounts receive 5 credits; Auth may add signup friction (email confirmation / rate limits depending on Supabase project settings)	The app does not enforce one account per real-world person. Auth proves an account/session, not unique human identity. Duplicate license plates are allowed (shared vehicles) and are not used as anti-Sybil proof. We have not fully audited all production Auth settings as a strong identity control	Phone verification; one welcome grant per stronger identity; device/account signals; credit-loop anomaly detection; rate limits
GPS spoofing to claim	Distance check uses provided coords	Client can spoof coordinates	Server-side attested location (hard on web)
Claim spam / grieving	One active claim; distance; credits check	Other accounts can still claim after cancel	Stronger anti-abuse

Live-location / Realtime flood	Atomic DB rate limit (2s) on GPS snapshots and status broadcasts; strict sequence monotonicity; broadcast only after DB accept	Native HTTP 429 may briefly show “temporarily unavailable” UI; not a tracker stop	Server Action rate limits; CSP
Repeated cancel/reclaim	Same-seeker reclaim blocked after voluntary release	Other accounts can still claim	Stronger anti-abuse
Sharing plate suffixes out-of-band	Social problem	Cannot stop verbal sharing	Accept as residual risk
Public spot taken by non-user	Product limitation disclosed	Unavoidable for street parking	Clear UX expectations
Push/token abuse	Device tables + auth (optional pipeline)	Production push not fully verified; depends on Edge/config hardening	Treat as experimental; harden only if enabling push

Do not claim fraud or unique-identity problems are “solved.”

11. Security hardening (repository audit)

Implemented hardening based on a focused pre-deployment review. **Not** a full penetration test.

Dormant RPC lockdown

- `get_handoff_code(uuid)` is legacy/dormant (plate suffix verification is the product path).
- Migration `20260823100000_security_hardening.sql` **revokes EXECUTE from `authenticated`**. Prior migrations already revoked `PUBLIC` and `anon`. Clients cannot retrieve spoken codes even if the function body is empty.

Live-location atomic rate limiting

Web and native path: `handoff-seeker-location` Edge Function → service-role RPCs → Realtime Broadcast **only if accepted**. Clients do not publish GPS directly onto Realtime.

RPC	Purpose	Returns
<code>upsert_claim_live_location</code>	Latest GPS snapshot per claim	<code>accepted / stale_sequence / rate_limited</code>
<code>try_accept_claim_location_status</code>	Throttle seeker-location-status broadcasts	<code>accepted / rate_limited</code>

Rules (GPS snapshots):

1. Lock `public.claims` row FOR UPDATE (serializes all writers, including first insert).
2. Compare interval using `pg_catalog.clock_timestamp()` (wall clock), not transaction-start `now()`.
3. **Strict sequence:** `incoming_sequence <= stored_sequence` → `stale_sequence`. Client `sentAt` is stored but does **not** override sequence ordering.
4. **Rate limit:** minimum **2 seconds** between accepted updates (below native ~3s send interval; leaves jitter margin).
5. Rejected updates are **never broadcast** to the publisher.

Status events (`paused` / `stopped`) use a lightweight throttle table `claim_live_status_throttle` (one row per claim; deleted on terminal claim). Same 2s floor; no GPS persistence required.

Edge Function error sanitization

Postgres/PostgREST `detail` strings are not returned to clients on snapshot/broadcast failures (generic error codes only).

HTTP response headers (Next.js)

`next.config.ts` sets baseline headers on `/:path*`:

- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-Frame-Options: DENY`
- `Permissions-Policy: camera=(), microphone=(), geolocation=(self)`

`geolocation=(self)` **allows** geolocation on the app origin (required for map, claim distance, live location). It is **not** `geolocation=()` (which would disable it). **CSP is not implemented** (deferred).

Auth redirect hardening

`src/lib/auth/safe-redirect.ts` rejects open redirects: requires leading `/`, blocks `//`, `://`, backslashes, and control characters.

Native / diagnostic token logging

Android/iOS native POST paths log `accessTokenPresent=true/false` only — not JWT values. JS diagnostics (`log-handoff-live.ts` , `log-handoff-live-receiver.ts`) strip token-related keys before logging.

12. Remaining security limitations

- Publishable keys are in the browser (normal for Supabase; RLS must hold).
- No perfect anti-collusion.
- Completion does not require proximity proof.
- Spoken-code table remnants exist but are not the UX; still used for attempt/lock state.
- Push infrastructure is optional/experimental and not part of the verified core web MVP.
- Native/Edge paths add operational secret-handling requirements if enabled.
- No Content-Security-Policy yet.
- Server Actions do not have explicit rate limiting (Auth email send rate limits are Supabase-side).

- Confirm-email anti-enumeration can show Check your email even when no mail is sent for an already-registered address — by design; UX is neutral, not a hard existence check.
 - Live-location 429 responses may briefly flicker “temporarily unavailable” UI without stopping native tracking.
-

13. Future security improvements (NOT CURRENTLY IMPLEMENTED)

- Stronger bot/account signup controls / phone verification
 - Proximity-gated completion
 - Anomaly detection on credit loops / repeated pairs
 - Content-Security-Policy
 - Server Action rate limits
 - Formal penetration test
 - Remove dormant spoken-code fields if no longer needed
 - Terminal parking-spot Realtime grace window (publisher map)
-

Repository references

- `src/proxy.ts` , `src/lib/supabase/proxy.ts`
- `src/actions/auth.ts`
- `src/app/auth/callback/route.ts`
- `src/app/(auth)/forgot-password/page.tsx`
- `src/app/auth/reset-password/page.tsx`
- `src/lib/auth/email-verification.ts`
- `src/lib/auth/password-recovery.ts`
- `src/lib/auth/password-policy.ts`
- `src/lib/auth/vehicle-access.ts`
- `src/lib/auth/safe-redirect.ts`
- `src/lib/auth/post-auth-redirect.ts`
- `src/lib/feedback/error-map.ts`
- `src/lib/validations/*`
- `src/lib/location/use-seeker-live-location-share.ts` , `fetch-claim-live-location.ts`
- `.env.example`
- `supabase/migrations/20260802111257_auth_profile_and_rls.sql`
- `supabase/migrations/20260819130000_prevent_seeker_reclaim.sql`
- `supabase/migrations/20260818170000_publisher_verifies_seeker_plate.sql`
- `next.config.ts` (security headers)
- `supabase/functions/handoff-seeker-location/index.ts`
- `src/lib/security/security-hardening.test.ts`
- `src/lib/location/claim-live-location-rate-limit.test.ts`
- `supabase/migrations/20260823100000_security_hardening.sql`
- `supabase/migrations/20260823110000_claim_live_location_atomic_rate_limit.sql`
- `supabase/migrations/20260823120000_claim_live_location_rate_limit_hardening.sql`
- `supabase/migrations/20260817140000_claim_live_locations_snapshot.sql`
- `README.md`